

作业五实验报告

王琳睿 241300009 (email: 1637928975@qq.com)

(南京大学 人工智能学院, 南京 210093)

1 学习方法介绍

1.1 A2C算法（策略梯度算法）介绍：

A2C 算法的全称为 Advantage Actor-Critic 算法，是一种基于 Actor-Critic 框架的强化学习算法，它结合了策略梯度方法（Actor）和价值函数估计方法（Critic）的优点，通过同时优化策略和价值函数来提高学习效率和性能。A2C 算法通过并行化多个环境来加速训练过程，并使用多个策略梯度更新来稳定训练过程。此外，A2C 算法还引入了熵正则化项来鼓励策略的探索性，从而提高算法的泛化能力。

算法基础：

1、**策略梯度算法**：策略梯度算法（Policy Gradient, PG）是一类直接对策略进行优化的强化学习算法。其核心思想是通过梯度上升法来调整策略参数，使得策略在选择高奖励动作时的概率增加，从而最大化累积奖励。

算法流程：

采样轨迹：用当前策略 π_{θ} 与环境交互，收集一条完整轨迹 $\tau = (s_0, a_0, r_0, \dots, s_T, a_T, r_T)$ 。

计算累积奖励：对每条轨迹计算折扣累积奖励 $G_t = \sum_{k=t}^T \gamma^k r_k$ （从时刻 t 到结束的总奖励）。

计算梯度：对每个时刻 t ，计算损失 $L_t = -\log \pi_{\theta}(a_t | s_t) / G_t$ （负号将梯度上升转为梯度下降优化）。

更新参数：对所有时刻的损失求和，反向传播更新策略参数 θ 。

2、**Actor-Critic 算法**：结合了策略梯度方法（Policy Gradient）和值函数估计，核心是通过 Actor（策略函数）选择动作，通过 Critic（值函数）评估这些动作，并相互协作改进。目的是利用 Critic 降低策略梯度的方差，同时保留策略方法的灵活性。

A2C 的优化：

引入优势函数，其物理意义是：“当前动作相对于平均动作的优劣程度”，通过减去状态价值，将绝对价值转化为相对优势，避免策略被局部高回报动作过度吸引。

通过优势函数平衡策略梯度的方差与偏差。

1.2 MCTS介绍：

作用蒙特卡洛树搜索（MCTS）是一种用于决策过程的算法，特别适用于博弈类问题。它通过模拟大量的随机游戏来估计每个可能动作的价值，从而选择最优的下一步。MCTS 算法的

核心思想是将计算资源集中在最有潜力的分支上，从而提高搜索效率。

MCTS 的四个步骤：

选择（Selection）：从根节点开始，根据 UCB（Upper Confidence Bounds）公式递归选择最优的子节点，直到到达一个叶子节点。UCB 公式如下：

$$UCB1(S_i) = V_i + c * \sqrt{\log(N) / n_i}$$

扩展（Expansion）：如果当前叶子节点不是终止节点，则创建一个或多个子节点，并选择其中一个进行扩展。

模拟（Simulation）：从扩展节点开始，运行一个模拟，直到游戏结束。模拟的结果用于评估该节点的价值。

回溯（Backpropagation）：使用模拟的结果，反向传播以更新当前动作序列的节点信息。

1.3 DQN介绍：

框架代码中涉及到 DQN，但本次作业我未使用，简单重述一下大作业 4 中对其的介绍：

4.1 dqn（Deep Q-Network）介绍：

Deep Q-Network，即“深度 Q 网络”，是将深度学习与 Q 学习相结合的强化学习算法，主要用于解决传统 Q-learning 高维状态空间的局限性问题。

Q-Learning：传统的 Q 学习算法，使用表格存储 Q 值。

Q：代表 Q 函数（Q-function），即动作价值函数，表示在给定状态下执行某个动作的长期期望回报。

使用神经网络来近似 Q 函数，通过监督学习的方式训练 Q 网络。

2 MCTS 实验结果

MCT 代码：

```
class MCTSNode: 5 用法  Arike
    """MCTS 树节点"""
    def __init__(self, position, player, parent=None, prior=0.0):...

    def expanded(self): 1 个用法 (1 个动态)  Arike
        return len(self.children) > 0

    def is_terminal(self): 1 个用法 (1 个动态)  Arike
        return self.position.is_game_over()

    def value(self): 1 个用法 (1 个动态)  Arike
        if self.visit_count == 0:
            return 0
        return self.value_sum / self.visit_count
```

```

class MCTS: 8 用法 吕 Arike
    """ 蒙特卡洛搜索 """
    def __init__(self, exploration_weight=1.0, simulation_limit=20):...

    def _select(self, node):...

    def _expand(self, node):...

    def _simulate_random(self, node):...

    def _simulate(self, node):...

    def _backpropagate(self, node, value):...

    def search(self, position, current_player, num_simulations=50):...

    def get_best_action(self, position, current_player, num_simulations=50, temperature=1.0):...

    def update_root(self, action):...

```

```

class AlphaGoMCTS(MCTS): 6 用法 吕 Arike
    """ 整合策略网络的AlphaGo MCTS """
    def __init__(self, deep_policy_agent, rollout_policy_agent, 吕 Arike
                 exploration_weight=1.0, simulation_limit=50):...

    def _expand(self, node):...

    def _simulate(self, node):...

    def _evaluate_position(self, position, player):...

```

MCTS 为框架，使用随机 MCTS；AlphaGoMCTS 为可加载已保存模型的 MCTS。
结果：

对弈统计结果

总对局数：10

神经网络MCTS胜场：2

随机MCTS胜场：8

平局：0

随机MCTS获胜

清理资源...

对弈统计结果

总对局数：50

神经网络MCTS胜场：10

随机MCTS胜场：35

平局：5

清理资源...

所有测试完成！

训练模型后能够正常加载整合和运行，但是较随机 MCTS 相比胜率较低。

3 对手池方法

3.1 对手池方法介绍：

“对手池”（Opponent Pool）是一个博弈论、游戏 AI 及策略分析中的概念，指在对抗性场景中，用来训练、测试 AI 或制定策略时所考虑的、具有不同风格、技能水平和行为模式的潜在或实际对手群体，其目的是为了构建更具适应性、更全面的应对策略，避免 AI 只面对单一类型对手而产生的短板。

对手池中有关方法：

```
class OpponentPool: 2 用法  Arike *
    def __init__(self, pool_dir="./opponent_pool", max_size=10, board_size=5):...

    def load_pool(self):...

    def save_pool(self):...

    def add_opponent(self, agent, model_type="deep", name=None, win_rate=None):...

    def add_double_agent(self, deep_agent, rollout_agent, name=None, win_rate=None):...

    def _remove_weakest(self):...

    def get_opponent(self, strategy="balanced", require_rollout=False):...

    def load_agent(self, name, model_type="deep"):...

    def load_double_agent(self, name):...

    def update_elo(self, agent1_name, agent2_name, result, k=32):...

    def get_pool_info(self):...

    def print_pool_status(self):...
```

3.2 对手池方法实验结果：

训练完成后的 AI vs 用均匀随机 rollout 的 MCTS AI 实验结果：

对弈统计结果	=====
总对局数: 10	总对局数: 50
神经网络MCTS胜场: 3	神经网络MCTS胜场: 16
随机MCTS胜场: 6	随机MCTS胜场: 31
平局: 1	平局: 3
清理资源...	随机MCTS获胜
	清理资源...

4 调参及结果

由于时间原因，本次只运行了贝叶斯优化，得到以下结果：

```
{
  "score": 52.0,
  "params": {
    "mcts_simulations": 124,
    "exploration_weight": 1.6195872025782099,
    "rollout_limit": 51,
    "critic_lr": 0.006010603698064649,
    "pi_lr": 0.0003978067187448131,
    "entropy_cost": 0.0491950973167859,
    "batch_size": 28,
    "num_critic_before_pi": 7
  },
  "metrics": {
    "train_win_rate": 0.35,
    "win_rate_no_mcts": 0.7333333333333333,
    "win_rate_mcts": 0.2,
    "mcts_improvement": 0.0
  },
  "timestamp": "2026-01-03T20:27:00.832076"
}
```

结果：

对弈统计结果	对弈统计结果
总对局数: 10	总对局数: 50
神经网络MCTS胜场: 5	神经网络MCTS胜场: 16
随机MCTS胜场: 5	随机MCTS胜场: 25
平局: 0	平局: 9
双方平手	清理资源...
清理资源...	所有测试完成!

模型表现有明显的提升。

5 Reference

- [1] [A2C\(Advantage Actor-Critic\) 算法 a2c 算法 -CSDN 博客](https://blog.csdn.net/weixin_55749690/article/details/141394790)
https://blog.csdn.net/weixin_55749690/article/details/141394790
- [2] [现代强化学习 - 策略梯度算法学习 | vortezwohl](https://vortezwohl.github.io/rl/2025/03/26/%E7%AD%96%E7%95%A5%E6%A2%AF%E5%BA%A6%E7%AE%97%E6%B3%95.html)
<https://vortezwohl.github.io/rl/2025/03/26/%E7%AD%96%E7%95%A5%E6%A2%AF%E5%BA%A6%E7%AE%97%E6%B3%95.html>
- [3] [一文详解策略梯度算法 \(REINFORCE\) — 强化学习 \(8\) -CSDN 博客](https://blog.csdn.net/wh1236666/article/details/149539263)
<https://blog.csdn.net/wh1236666/article/details/149539263>